MODULE 2
GIT

List of Topics

1. Introduction to VCS and GIT
2. GIT Installation
3. GITHUB Account Creation
4. GIT Configuration
5. GIT Basics
6. Working with Branches
7. Remote Repository
8. GIT Collaboration
9. Undo Changes
10. GIT Tools

Introduction to Version Control System (VCS)

- A Version Control System (VCS) is a software tool used to track changes made to files and source code over time.
- It helps developers maintain different versions of a project and understand what changes were made.
- VCS is especially useful when multiple developers work on the same project because it helps them manage changes, collaborate, and recover earlier versions when required.

Why Version Control is Needed

Without version control, developers may create many copies of the same project, such as:

- project_final
- project_final_new
- project_final_latest

This approach makes it difficult to identify which version is correct and who made a particular change.

A VCS provides a systematic way to:

- Track changes in source code.
- Maintain the history of a project.

- Identify who made a change.
- Restore an earlier version when required.
- Allow multiple developers to work on the same project.
- Create separate development paths using branches.

Types of Version Control Systems

Type	Description	Example
Local VCS	Version history is maintained on the local computer.	RCS
Centralized VCS	A central server stores the main version history.	SVN
Distributed VCS	Each developer can have a complete repository with its history.	GIT

What is GIT

- GIT is a distributed version control system used to track changes in source code and manage different versions of a project.
- GIT was originally created by Linus Torvalds in 2005
- GIT is commonly used with platforms such as GitHub, GitLab, and Bitbucket for remote repository hosting and collaboration. (GitHub is preferable)

Why is GIT Used

- To maintain the history of source-code changes.
- To work on different features separately using branches.
- To compare changes between versions.
- To restore previous versions of files.
- To support collaboration between developers.
- To work locally and synchronize changes with remote repositories.

Features of GIT

- Distributed – Each developer can have a complete local repository.
- Version History – Changes can be recorded and reviewed.
- Branching – Developers can create separate branches for features or fixes.
- Merging – Changes from different branches can be combined.
- Collaboration – Multiple developers can work on the same project.
- Data Integrity – GIT uses checksums to identify and protect stored content.